SimTalk Access to 3D Functions

SimTalk Access to 3D Functions

Plant Simulation provides general *SimTalk* functions plus a number of specific functions for accessing the *3D* objects.

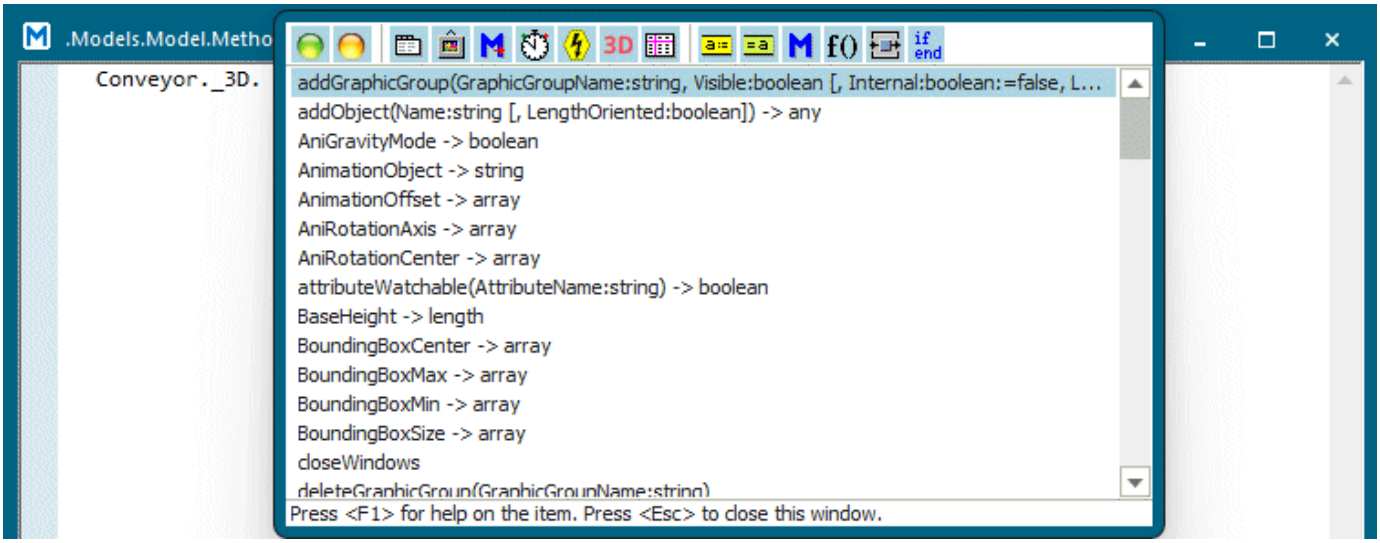
Note

SimTalk normally does not distinguish between upper- and lower-casing for the names of *methods*, *attributes* and *read-only attributes*, which you type into the source code of your **Method [object]**.

You can view and access the **attributes and methods** of the 3D objects by clicking **Auto Complete** on the **Edit** ribbon tab of the *Method Editor*.

Note

To access the **attributes and methods** of the 3D objects, type in `_3D`, followed by a period like so `_3D.` in you instructions.



Most of the *3D attributes* and *3D methods* listed in the table of contents use the **array** data types for setting and getting values.

See also

3D Properties Visualizing the Simulation

Copy Object Settings to a Method

General Access to SimTalk

See also, General Access

__Accessing the 3D Window

__Accessing View Options

__Accessing the Bounding Box

__Accessing Length-oriented Objects

__Accessing the Appearance of Objects and MUs

__Accessing the Appearance of the Store

__Accessing the Background Color of Frame and Folder

__Accessing Object Captions

__Accessing the Fill Level of the Buffer

__Accessing Objects in 3D

__Accessing Point Clouds

__Accessing Transformation Settings

See also, Accessing Animations

__Accessing MU Animations

__Accessing Self Animations

__Accessing Camera Animations

__Accessing Robot Arm Animations

__Accessing Joints

__Accessing Poses

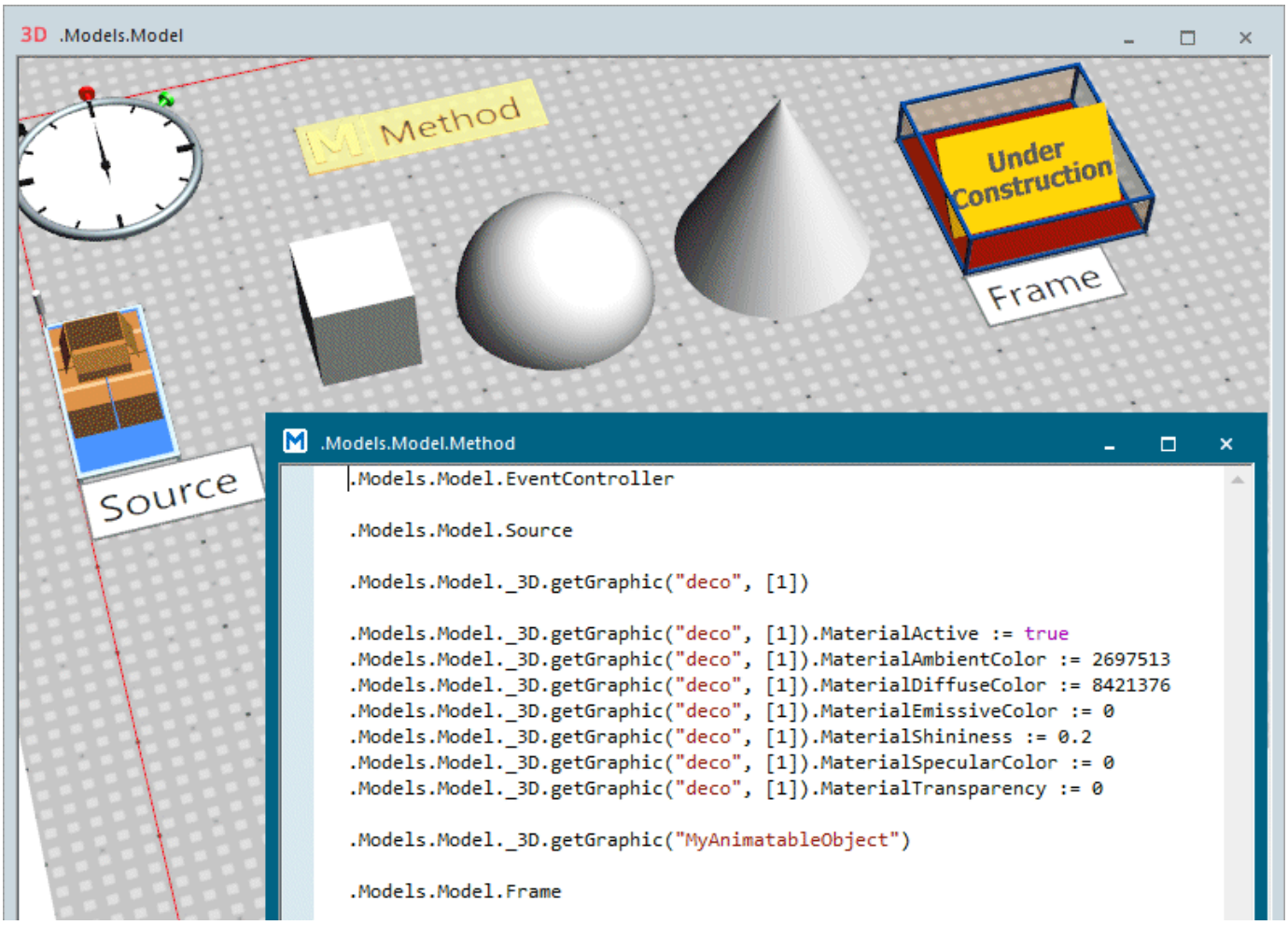
__Accessing the Worker

See also, Accessing Graphics

_Accessing Methods of Graphic Shapes

Copy Object Settings to a Method

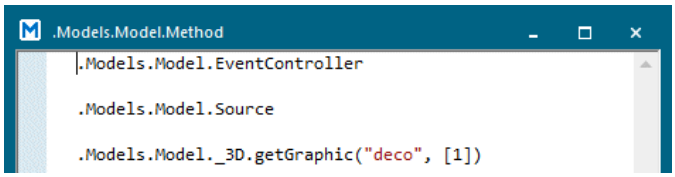
You can copy the following settings of simulation objects, of the animatable objects and of graphics and paste them into a *Method*:

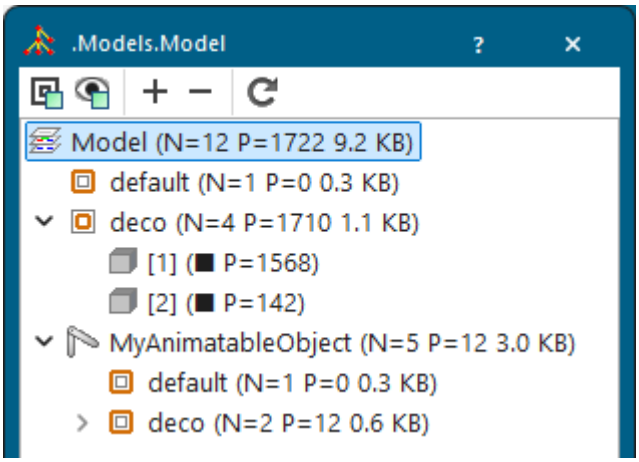


- To copy the **path of the selected object**, press **Ctrl+C** in the *Frame*. Then change to the *Method* and press **Ctrl+V** to paste the path. This is how we proceeded with the objects *EventController*, *Source*, and *Frame* in the example.
- To copy the **path of the selected shape**, press **Ctrl+C** in the *Frame*. Then change to the *Method* and press **Ctrl+V** to paste the path. This is how we proceeded with the *Cuboid* in the example.
- To copy the **current material settings** of the selected shape, press the **Spacebar** in the *Frame* and change to the tab **Material**. Activate the **Material** and click  , to copy the current material settings. Then change to the *Method* and **Ctrl+V** to paste the material settings. This is how we proceeded with the *Cuboid* in the example.
- To copy the **path of the selected animatable object**, press **Ctrl+C** in the *Frame*. Then change to the *Method* and press **Ctrl+V** to paste the path. This is how we proceeded for the *Sphere* in the example.

- You can then type in a period and press **Ctrl+Spacebar** to extend the path, compare the command **Auto Complete**.

In the illustrations of the *Method* the number in brackets, in the example below the highlighted `("deco", [1])`, designates the graphic node with the respective number. The dialog **Show Graphic Structure** also shows this number.





See also

_3D.getGraphic [SimTalk]

Auto Complete

Show Graphic Structure [context menu]

Accessing PLMXML Kinematics Files

_Accessing PLMXML Kinematics Files

SimTalk provides the functions listed in the table of contents to the left for accessing PLMXML kinematics files.

applyPLMXMLKinematicStructure [SimTalk]

Applies a kinematics structure from the specified PLMXML file to the graphic designated by <Path>.

Remarks

During this process animatable objects are created within the simulation object or animatable object which are furnished with kinematics settings from the PLMXML file and which contain parts of the graphic designated by <Path>.

Type

Method

Syntax

```
<Path>.applyPLMXMLKinematicStructure(FilePath:string[,
PreferFirstEntryPoint:boolean:=true, PreferredEntryPointName:string:=""])
-> boolean
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* designates the path to the PLMXML file containing the desired information.
- The optional parameter *PreferFirstEntryPoint* of data type *boolean* specifies if the first (`true`) or the last (`false`) of the potential kinematics structures is to be used.

  Default Value of the Parameter

  The default value is `true`.
- The optional parameter *PreferredEntryPointName* of data type *string* sets the name of the root of the kinematics structure which is to be used if several potential kinematics structures exist. If this name is an empty string `""`, the method ignores the parameter.

  If you specify both optional parameters, the preferred name is stronger than the preferred first or last entry point.

  Default Value of the Parameter

  The default value is an empty string `""`.

Return Value

The return value has the data type *boolean*.

true if a kinematics structure was found and applied.

false if no kinematics structure was found.

Examples

```
var newGraphic: any :=
MyStation._3D.getGraphic("default").importGraphics("D:\Graphic.jt")
var kinematicsApplicable:boolean :=
newGraphic.applyPLMXMLKinematicStructure("D:\kin.plmxml")
if not kinematicsApplicable then
   -- error handler
end
```

```
var roots: string[] := getPotentialPLMXMLKinematicRoots("D:\kin.plmxml")
if roots.Dim = 0 then
   -- error handler
else
   var newGraphic: any :=
MyStation._3D.getGraphic("default").importGraphics("D:\Graphic.jt")
   if roots.Dim > 1 then
   var kinematicsApplicable: boolean  :=
newGraphic.applyPLMXMLKinematicStructure("D:\kin.plmxml", true, roots[2])
   if not kinematicsApplicable then
       -- error handler
   end
       else
       var kinematicsApplicable: boolean  :=
newGraphic.applyPLMXMLKinematicStructure("D:\kin.plmxml")
       if not kinematicsApplicable then
-- error handler
       end
   end
end
```

See also

getPotentialPLMXMLKinematicRoots [SimTalk]

getPotentialPLMXMLKinematicRoots [SimTalk]

Reads kinematics information from the designated PLMXML file and lists all roots of all potential kinematics structures from the file.

Type

Method

Syntax

```
getPotentialPLMXMLKinematicRoots(FilePath:string) -> string[]
```

Parameter

The parameter *FilePath* of data type *string* designates the path to the PLMXML file containing the desired information.

Return Value

The return value is an *array* of data type *string*.

- If the *array* is empty, *Plant Simulation* did not find a kinematics root. This can have different reasons, for example a PLMXML file that does not contain any kinematics information or whose kinematics are incomplete.
- If the *array* contains a single name, the file contains a unique kinematics root.
- If the *array* contains several names, several ways exist for interpreting the structure which are indistinguishable without context.

> **Note**
> The same name can occur several times.

Example

```
var roots : string[] := getPotentialPLMXMLKinematicRoots("D:\kin.plmxml")
print roots.dim, roots[1]
```

See also

applyPLMXMLKinematicStructure [SimTalk]

Accessing the 3D Window

__Accessing the 3D Window

SimTalk provides the *methods* and *functions* listed in the table of contents to the left for accessing the *3D window*.

_3D.closeWindows [SimTalk]

Closes all windows which are opened for the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.closeWindows
```

Example

```
MyStation._3D.closeWindows
```

See also

closeDialog [SimTalk] - all objects

F3DactivateCameraMark [SimTalk]

Activates the specified camera mark in the currently active 3D window.

Remarks

The camera mark changes the viewpoint and object of interest in the active 3D window.

Type

Function

Syntax

```
F3DActivateCameraMark(CameraMark:string)
```

Parameter

The parameter *CameraMark* of data type *string* designates the name of the camera mark.

Example

```
F3DactivateCameraMark("MyCameraMark")
```

See also

Camera Marks

F3DattachCamera [SimTalk]

Attaches the camera of the active 3D window to the specified object or detaches a previously attached camera in the active 3D window.

Remarks

F3DattachCamera will not do anything if there either is no 3D window open or if the active 3D window is shown in **Planning View**.

Type

Function

Syntax

```
F3DattachCamera(ObjectToAttach:any)
```

Parameter

The parameter *ObjectToAttach* of data type *any* specifies the object to which you want to attach the camera:

- Specify `void` to detach an attached camera in the active 3D window.
- Specify an object or a 3D object, for example an animatable object, to attach the camera of the active 3D window to that object.

Example

```
F3DattachCamera(.MUs.Part:1)
```

See also

Attach Camera

F3DconfigurePlanningView [SimTalk]

Sets the view to a **Planning View** designated by the specified parameters.

Type

Function

Syntax

```
F3DconfigurePlanningView(ViewPosition:real[2], Zoom:real[,
ObjectOfInterest:object/3Dobject])
```

Parameters

You can specify the following parameters:

- The parameter *ViewPosition* is an *array* of data type *real* with two values. They designate the position of the view's projection in the **XY Plane**.
- The parameter *Zoom* of data type *real* designates the zoom factor of the view.
- The optional parameter *ObjectOfInterest* of data type *object* opens the object in the active 3D window. The object of interest can be an object, a 3D object (for example an animatable object), or a simulation object.

  If you do not specify this parameter, the open 3D window remains unchanged.

Example

```
F3DconfigurePlanningView([-14, -15], 16, .Models.MyWarehouse)
```

See also

_3D.getObject [SimTalk]

Camera Marks

Use Planning View

F3DconfigureView [SimTalk]

Generate SimTalk Code and Copy to Clipboard [camera mark]

F3DconfigureView [SimTalk]

Sets the view to a **Modeling View** designated by the specified parameters.

Type

Function

Syntax

```
F3DconfigureView(CameraPosition:real[3], CameraRotationX:real,
CameraRotationZ:real[, ObjectOfInterest:object])
```

Parameters

You can specify the following parameters:

- The parameter *CameraPosition* is an *array* of data type *real* with three values. They designate the position of the camera in three-dimensional space.
- The parameter *CameraRotationX* of data type *real* designates the rotation of the camera on the x-axis.
- The parameter *CameraRotationZ* of data type *real* designates the rotation of the camera on the z-axis.
- The optional parameter *ObjectOfInterest* of data type *object* opens the object in the active 3D window. The object of interest can be an object, a 3D object (for example an animatable object), or a simulation object.

  If you do not specify this parameter, the open 3D window remains unchanged.

Example

```
F3DconfigureView([-8.129, -3.707, 7.832], 45.000,
-15.000, .Models.MyWarehouse)
```

See also

_3D.getObject [SimTalk]

Camera Marks

F3DconfigurePlanningView [SimTalk]

Generate SimTalk Code and Copy to Clipboard [camera mark]

Accessing View Options

__Accessing View Options

SimTalk provides the *attributes* listed in the table of contents to the left for setting view options of the *Frame* in *3D*.

See also

View Ribbon Tab

_3D.PlanningView [SimTalk]

Activates (`true`) or deactivates (`false`) the **Planning View** in the *Frame* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.PlanningView:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.PlanningView := true
```

See also

View Ribbon Tab > **Use Planning View**

_3D.ShowBasePlate [SimTalk]

Shows (`true`) or hides the *base plate* (`false`) below the grid in the *Frame* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.ShowBasePlate:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowBasePlate := false
```

See also

View Ribbon Tab > **Show Base Plate**

_3D.ShowConnections [SimTalk]

Shows *Connectors/Interfaces/Markers* in the active *Frame* window designated by <Path> (`true`) or hides them (`false`).

Remarks

_3D.ShowConnections shows or hides *material flow directions* of the length-oriented objects for which you set *_3D.ShowMaterialFlowDirection* to `true`.

Type

Attribute

Syntax

```
<Path>._3D.ShowConnections:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowConnections := false
```

See also

View Ribbon Tab > **Show Connections**

Show Material Flow Direction [check box]

_3D.ShowMaterialFlowDirection [SimTalk]

View Ribbon Tab > **Show Material Flow Directions [button]**

_3D.ShowMaterialFlowDirections [SimTalk]

_3D.ShowExternalGraphics [SimTalk]

Shows (`true`) or hides (`false`) *external graphic groups* of the *Frame* designated by <Path> inward, i.e. in 3D windows that opened the designated *Frame*.

Type

Attribute

Syntax

```
<Path>._3D.ShowExternalGraphics:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowExternalGraphics := false
```

See also

View Ribbon Tab > **Show External Graphic Groups**

External [graphic group]

_3D.ShowGrid [SimTalk]

Shows (`true`) or hides (`false`) the *grid* in the *Frame* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.ShowGrid:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowGrid := false
```

See also

View Ribbon Tab > **Show Grid**

_3D.ShowLabels [SimTalk]

Shows (`true`) or hides (`false`) the *labels* of the objects in the *Frame* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.ShowLabels:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowLabels := false
```

See also

View Ribbon Tab > **Show Object Names**

_3D.ShowMaterialFlowDirections [SimTalk]

Shows (`true`) or hides (`false`) the *material flow direction arrows* of the length-oriented objects in the *Frame* designated by <Path> .

Type

Attribute

Syntax

```
<Path>._3D.ShowMaterialFlowDirections:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowMaterialFlowDirections := true
```

See also

View Ribbon Tab > **Show Material Flow Directions [button]**

View Ribbon Tab > **Show Connections**

Show Material Flow Direction [check box]

_3D.ShowNames [SimTalk]

Shows (`true`) or hides (`false`) the *names* of the objects in the *Frame* designated by <Path> .

Type

Attribute

Syntax

```
<Path>._3D.ShowNames:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowNames := false
```

See also

View Ribbon Tab > **Show Object Names**

_3D.ShowPointClouds [SimTalk]

Shows (`true`) or hides (`false`) *point clouds* in the *Frame* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.ShowPointClouds:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowPointClouds := false
```

See also

View Ribbon Tab > **Show Point Clouds**

_3D.ShowShadows [SimTalk]

Shows (`true`) or hides (`false`) *shadows* in the *Frame* designated by <Path> .

Remarks

Plant Simulation only shows the shadows of the objects in a *Frame*.

Captions, manipulators, path visualizations, and some display objects, for example *Variable*, *Comment*, etc. do not throw a shadow most of the time.

Type

Attribute

Syntax

```
<Path>._3D.ShowShadows:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowShadows := false
```

See also

View Ribbon Tab > **Show Shadows**

_3D.ShowSky [SimTalk]

Shows (`true`) or hides (`false`) the *sky* in the *Frame* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.ShowSky:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowSky := true
```

See also

View Ribbon Tab > **Show Sky**